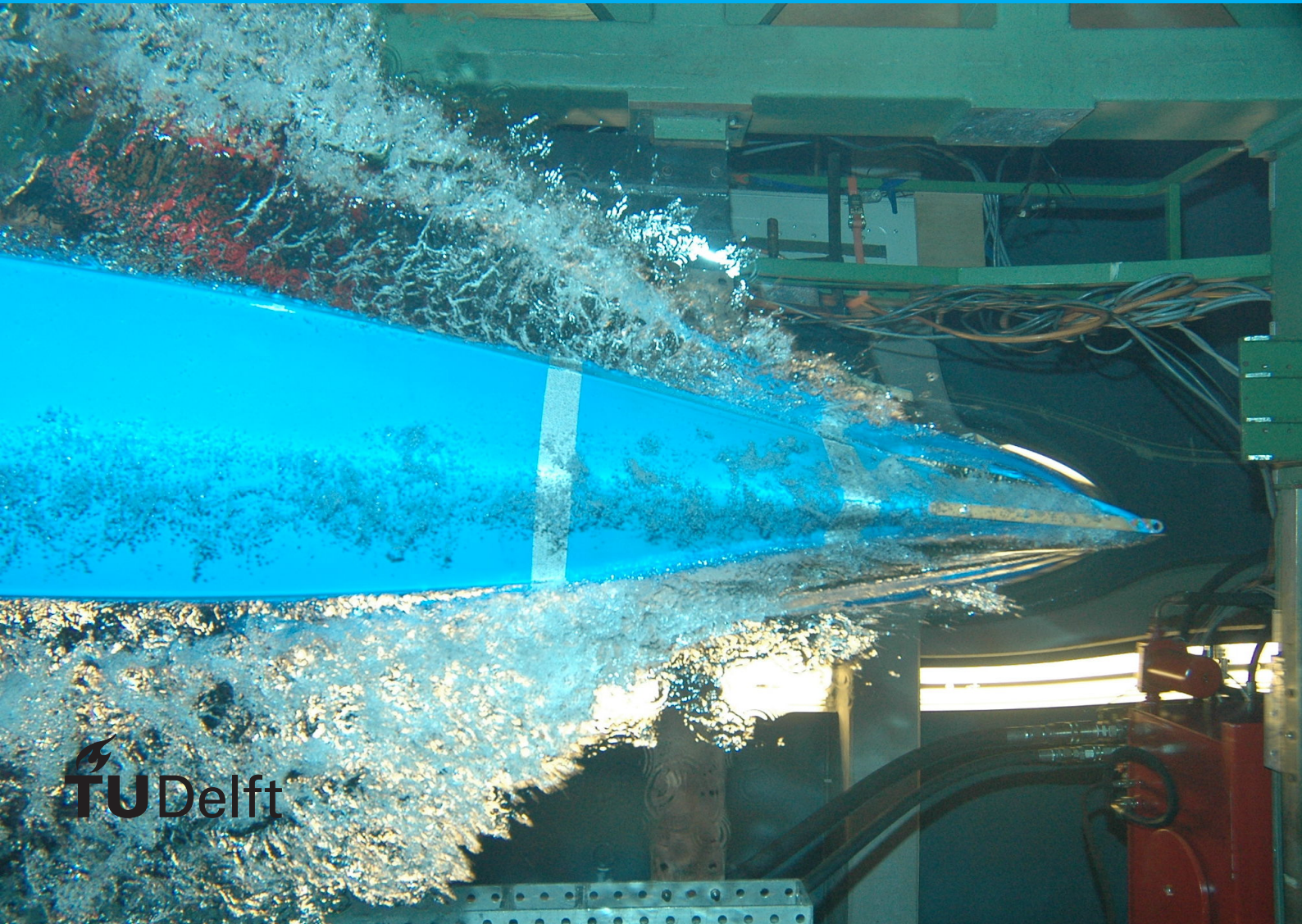
Policy Gradient Reinforcement Learning for quadrotor control

Preliminary thesis

A. M. de Windt



Policy Gradient Reinforcement Learning for quadrotor control

Preliminary thesis

by

A. M. de Windt

to obtain the degree of Master of Science

at the Delft University of Technology,

to be defended publicly on Tuesday January 1, 2013 at 10:00 AM.

Student number:	1234567	
Project duration:	March 1, 2012 – January 1, 2013	
Thesis committee:	Prof. dr. ir. J. Doe,	TU Delft, supervisor
	Dr. E. L. Brown,	TU Delft
	Ir. A. Aaronson,	Acme Corporation

This thesis is confidential and cannot be made public until December 31, 2013.

An electronic version of this thesis is available at `http://repository.tudelft.nl/`.

TU Delft

Todo list

Rewrite this paragraph so it's easier to read. 1
Expand a bit more here. 1
Add examples of actual use cases and research instead of a simple, ranging from ... to 1

Preface

Preface…

A. M. de Windt
Delft, January 2013

List of Figures

2.1 Example reward distribution for a 5-armed bandit problem. 4
2.2 Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies. 5

List of Tables

Acronyms

MDP Markov Decision Process.

RL Reinforcement Learning.

Symbols

G_t return (cumulative discounted reward) following time t.

$G_t^{\lambda a}$ λ-return, corrected by estimated action values.

$G_t^{\lambda s}$ λ-return, corrected by estimated state values.

$G_{t:h}$ flat return (uncorrected, undiscounted) from $t+1$ to h.

$G_{t:h}^{\lambda a}$ truncated, corrected λ-return, with action values.

$G_{t:h}^{\lambda s}$ truncated, corrected λ-return, with state values.

Q, Q_t array estimates of action-value function q_π or q_*.

V, V_t array estimates of state-value function v_π or v_*.

$\in$ is an element of, e.g., $s \in \mathcal{S}$.

$|\mathcal{S}|$ number of elements in $\mathcal{S}$.

$\mathcal{A}$ set of all actions.

$\mathcal{R}$ set of all possible rewards, a finite subset of $\mathbb{R}$.

$\mathcal{S}$ set of all states.

$\mathcal{S}^+$ set of all states, including the terminal state.

$\subset$ subset, e.g., $\mathcal{R} \subset \mathbb{R}$.

a an action.

$p\left(s', r | s, a\right)$ probability of transition to state s' with reward r, from state s and action a.

$p\left(s' | s, a\right)$ probability of transition to state s', from state s and action a.

$q_*(s, a)$ value taking action a in state s under the optimal policy.

$q_\pi(s, a)$ value taking action a in state s under policy π.

r a reward.

$r\left(s, a, s'\right)$ expected immediate reward on transition from s to s' under action a.

s a state.

$v_*(s)$ value of state s under optimal policy.

$v_\pi(s)$ value of state s under the optimal policy π (expected return).

Greek symbols

π policy, decision-making rule.

$\pi(a|s, \boldsymbol{\theta})$ probability of taking action a in state s given parameter $\boldsymbol{\theta}$.

$\pi(a|s)$ probablity of taking action a in state s under *stochastic* policy π.

$\pi(s)$ action taken in state s under *deterministic* policy π.

Contents

List of Figures vii

List of Tables ix

1 Introduction 1

2 Basics of Reinforcement learning 3

2.1 Finite Marcov Decision Process . . . 3

2.2 Elements of Reinforcement Learning . . . 3

2.3 k-armed bandit problem . . . 4

2.4 ϵ-greedy policies . . . 4

3 Tabular Methods 7

4 Approximate solutions 9

5 Past, present and future of Reinforcement Learning 11

6 Proposed thesis research 13

7 Conclusion 15

Bibliography 17

1 Introduction

The technological advancements and the steadily increasing demand for air and space travel of the past decades have lead to the development of new, more complex aerospace systems, resulting in new challenges in the field of aerospace control. At the same time, advancements in automated control have been seen in many domains, ranging from industrial manufacturing and household appliances to the rapid widespread adoption of UAVs. However, most of these systems are only able to complete their tasks within the narrow, predictable conditions they where designed for. Autonomous control systems need to be able to adapt to unexpected situations and environments. This has motivated researchers to search for new innovative methods that can be used to solve these new problems.

Rewrite this paragraph so it's easier to read.

One of the leading causes of fatal air traffic accidents is loss of control. If fully autonomous UAV's are to be used in the future, advanced flight control systems capable of adapting to rapidly changing conditions are necessary. This is because survival after mayor failure can only only be guaranteed if stable flight can be achieved.

Expand a bit more here.

Furthermore, thanks to the miniaturization of technologies, Micro Air Vehicles are becoming smaller, more diverse and are seeing a rise is use. However, these types of vehicles come with their own new problems. In order to design a reliable, well performing control system, an accurate dynamics model is needed. This is not always possible for MAVs because of the complex nature of MAV flight dynamics, inconsistent manufacturing and parameter adaptation during time. Because of their smaller size, MAVs are more susceptible to unforseen disturbances and have sensor and actuator limitations due to their weight and size requirements.

Adaptive control methods allow autonomous systems to reconfigure them self if necessary. This opens the possibility to create fault tolerant control systems and control systems capable of determining the governing dynamics and track changes that may occur over time due to degradation or varying flight conditions. These systems can also detect and compensate for the limited on-board resources and any inconsistencies that may have occurred during manufacturing.

Reinforcement learning is a framework of intelligent, self-learning methods that can be applied to a wide range of autonomous operations and applications. These methods have been used in robotics for decades and a lot of research has been taking place at the same time in a wide range of topics, from resource management in computer clusters to optimizing chemical reactions. However, most of these use cases involve taking high level decisions such as trajectories, schedules, etc, and not low level actuator control.

Add examples of actual use cases and research instead of a simple, ranging from ... to ...

RL methods are also relatively new in the field of aerospace and it's thus uncertain which systems can make effective use of the advantages of RL. This thesis proposes the development of online, self tuning low level quadcopters controllers. $\pi(a|s,\theta)$

The objective of this thesis is to investigate the effectiveness of Reinforcement Learning based control methods for low level quadrotor control.

What is Reinforcement Learning and what can it be used for?

How does Reinforcement Learning learning work?
What is Reinforcement Learning capable of?
How has Reinforcement Learning been used so far?
What state of the art technological developments make use of Reinforcement Learning?

Can Reinforcement Learning methods be used to improve the performance of low level control of a quadcopter?
What advantages do RL based controllers provide over classical control methods?
What RL methods can be effective when used as part of the low level control of a quadrotor?
What controller structures can make effective use of RL?
How do these new methods compare to classical control methods?
Are these RL based controllers fault tolerant and capable of adapting themselves in a practical environment?

2

Basics of Reinforcement learning

Reinforcement Learning (RL) is a method of machine learning different from other machine learning methods such as supervised learning which uses a set of labeled examples from a knowledgeable supervisor, and unsupervised learning which is typically used to find hidden patterns in unlabeled data. In Reinforcement Learning (RL) the learner is not told what action to take to achieve its goal, but instead it maps situations and actions by trying them out and discovering the most valuable. It does this by not only looking at the reward it received by it's latest action, but it also takes into account any future rewards it could receive in the future by taking that action.

2.1. Finite Marcov Decision Process

A Markov Decision Process (MDP) is a stochastic process that provides a mathematical framework used to describe problems where a learner learns from its actions to achieve an end goal. This framework is used in many areas, such as robotics, economics, manufacturing and in the case of this report, Reinforcement Learning.

The two main components of an Markov Decision Process (MDP) are the agent, which makes decisions, interacts with its environment and learns. And the environment which consists of everything else. The two are constantly interacting with each other, the agent will perform an action which changes the state of the environment and the environment responds by giving it a reward. The goal of the agent is to maximize the reward it receives from the environment.

This happens in a sequence of discrete time steps. At the beginning of each time step the environment is at a certain state s. The agent then decides based on its policy π which action a to take, this in turn changes the environment to the new state s' and the environment then gives the agent a certain reward r. In order for it to be a MDP this transition and the reward must satisfy the Marcov property, which means it needs to be memoryless and thus the environment's behavior must be independent of its past states. In other words, the probability of the environment to go from state s to s' and the reward r it gives back to the agent, must only be dependent on the initial state s and action a.

2.2. Elements of Reinforcement Learning

The main goal when solving an MDP problem is to find a policy π that defines the action $\pi(s)$ the agent will take in order to maximize the sum of rewards it receives. It characterizes the way in which the agent will behave at a given time by mapping the agents perceived state of the environment to the actions taken in those states. This policy may be as simple as a table or consist of several Neural Networks. The policy is the core of the agent in that it alone defines the behavior of the agent.

On each time step the environment will send the agent a reward. The goal of the agent is to maximize this reward in the long run. Thus this reward is used to tell the agent which states are good or bad. If the agent

receives a low reward for a certain action it can change the policy so it takes a different action in the future.

The reward signal tells the agent what the immediate effects of its action are. However, its goal is to maximize the cumulative sum of rewards it receives over time, and thus in order for it to receive higher rewards in the future, it might be necessary for the agent to take actions that yield lower rewards with the promise that it will receive higher rewards later. So the value is an indication of the reward the agent can expect to receive in the future by taking a certain action, and thus it is the main parameter used in the decision making process.

By defining the action taken during step t as A_t, and its corresponding reward as R_t we can define the true value of an arbitrary action a as:

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a] \tag{2.1}$$

If $q_*(a)$ was known, then the solution would be trivial. Just select the action with the highest value. So it's assumed that the action values are not known, but are instead estimated from the rewards the agent receives. The estimated value of action a at time t is known as $Q_t(a)$, thus $Q_t(a)$ needs to be as close to $q_*(a)$ as possible.

Finally some RL agents make use of a model for planning. This allows these agents to choose actions based on possible future situations before actually being experienced.

2.3. k-armed bandit problem

K-armed bandit problems will be used as examples in the following sections. These are simple learning problems where the agent is given the option to take k different action's. There is only one state and the episodes terminate after the agent has taken it's action. Furthermore the examples in this report use zero mean, unit variance normal distributions to select the true value $q_*(a)$ of the actions. The rewards given by the environment are then selected with $q_*(a)$ mean and unit variance normal distribution. Figure 2.1 shows an example reward distribution for a 5-armed bandit.

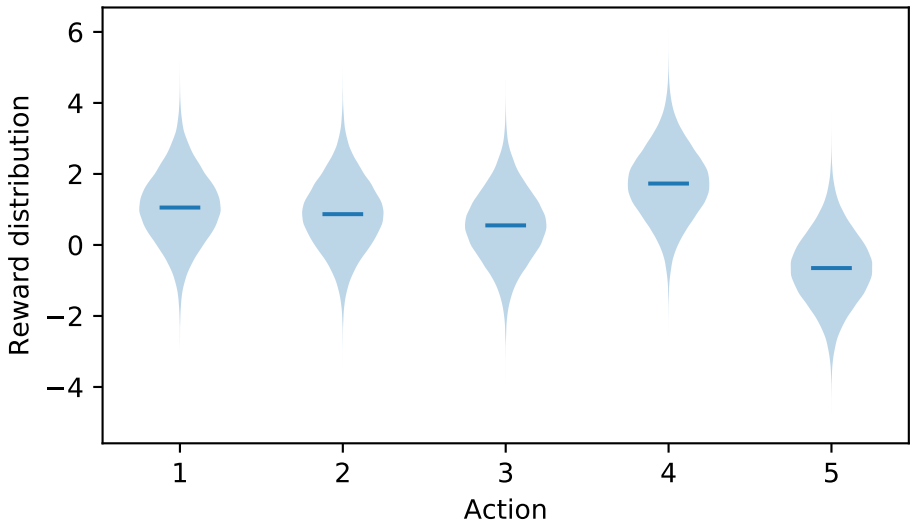


Figure 2.1: Example reward distribution for a 5-armed bandit problem.

2.4. ϵ-greedy policies

The main difference between Reinforcement Learning and other types of learning is that RL uses training information that evaluates the actions taken by the agent instead of a set of correct actions. This means that in order to determine $Q_t(a)$ the agent needs to explore and try out as many actions as possible in order to find

good behavior. To do this the agent can take random actions and learn from the rewards it gets back. This is called *exploration*.

However, the main goal of the agent is to maximize the rewards it receives from the environment over time. To achieve this goal the agent can choose to take the greedy actions. These are the actions with the highest value in $Q_t(a)$. This process of choosing the greedy action is known as *exploitation* since the agent is exploiting its knowledge to achieve its goal.

Unfortunately exploration and exploitation conflict with each other since in the former the agent wants to try out all possible actions, but only wants to perform a specific action if it's following a greedy policy. Thus a possible compromise is to use what is known as an ϵ-greedy policy. In these policies, the agent will choose a random action ϵ percentage of the time and the greedy action the rest of the time. This allows the agent to explore and try out new actions or old actions whose value might have changed, while also trying to achieve it's goal.

Since the agent is trying out new actions while following ϵ-greedy policies, it is able to learn faster and more than if it was following a greedy policy. To demonstrate this 2000 10-armed bandit problems are run for 1000 episodes. The average reward of all bandits are stored after each episode. Figure 2.2 shows the average reward and the percentage of agents that selected the optimal action for the greedy policy and two ϵ-greedy policies ($\epsilon = 0.1$ and $\epsilon = 0.1$). A full explanation of the learning method used can be found in the section ????, but in short, the action value function $Q_t(a)$ is estimated by averaging the rewards received from the environment.

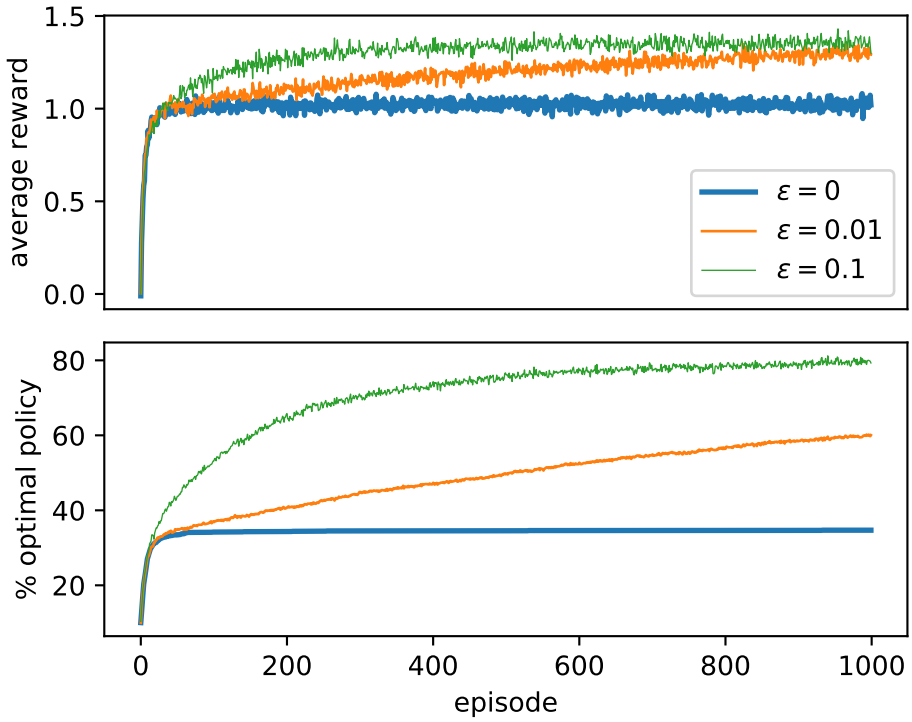


Figure 2.2: Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies.

As it can be seen when using ϵ-greedy policies the agents learn faster and are thus able to receive higher rewards sooner. On the other hand, while following the greedy policy, the agents will quickly choose the first action they find with a reward higher than the initial action value and then perform that action on all episodes. As a results many of the agents are not able to find the optimal policy.

3

Tabular Methods

4

Approximate solutions

5

Past, present and future of Reinforcement Learning

6

Proposed thesis research

7

Conclusion

Bibliography

[1] G. J. Alred, C. T. Brusaw, and W. E. Oliu. *Handbook of Technical Writing*. St. Martin's, New York, 2003 (seventh edition).

[2] Michel Goossens, Frank Mittelbach, and Sebastian Rahtz. *The LaTeX Companion*. Addison-Wesley, Reading, Mass., 1997.

[3] S. F. Gull. Developments in maximum-entropy data analysis. In J. Skilling, editor, *Maximum Entropy and Bayesian Methods*, pages 53–71. Kluwer Academic, Dordrecht, 1989.

[4] K. M. Hanson. Introduction to Bayesian image analysis. In M.H̃. Loew, editor, *Medical Imaging: Image Processing*, volume 1898 of *Proc. SPIE*, pages 716–731, 1993.

[5] L. Lamport. *LaTeX: A Document Preparation System*. Addison-Wesley, Reading, Mass., 1994.

[6] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equations of state calculations by fast computing machine. *J. Chem. Phys.*, 21:1087–1091, 1953.

[7] Frank Mittelbach, Michel Goossens, Johannes Braams, and David Carlisle. *The LaTeX Companion*. Addison-Wesley, Reading, Mass., 2004 (second edition).

[8] L. C. Perelman, J. Paradis, and E. Barrett. *Mayfield Handbook of Technical and Scientific Writing*. Mountain View, Mayfield, 1997. http://mit.imoat.net/handbook/.